

Newton-Raphson Method: MLE for the Location Parameter of a Cauchy Distribution

Sulagna Roy

September 13, 2026

Contents

1	Introduction	1
2	Step 1: Simulating Data from a Cauchy Distribution	2
3	Step 2: The Log-Likelihood Function	2
4	Step 3: Deriving the Score and the Hessian	3
5	Step 4: Implementing Newton-Raphson	3
6	Step 5: Visualizing the Log-Likelihood and the Iteration Path	5
7	Step 6: Numerical Results from This Run	7
8	Conclusion	7

1 Introduction

This document walks through a complete numerical example of using the **Newton-Raphson method** to find the **maximum likelihood estimate (MLE)** of the location parameter θ of a **Cauchy distribution**.

The Cauchy distribution is a classic textbook example in maximum likelihood theory because:

- Its log-likelihood function does **not** have a closed-form maximizer, so θ must be found numerically.
- The likelihood equation (the “score” equation) can have **multiple roots**, so a poor starting value for Newton-Raphson can converge to the wrong root, fail to converge, or even diverge.
- Because of these quirks, it is the standard example used to demonstrate why root-finding methods like Newton-Raphson must be applied carefully, with a sensible starting value and safeguards against numerical breakdown.

The probability density function of a Cauchy random variable X with location parameter θ and scale parameter $\gamma = 1$ is:

$$f(x \mid \theta) = \frac{1}{\pi [1 + (x - \theta)^2]}, \quad -\infty < x < \infty$$

Given an i.i.d. sample $x_1, x_2, \dots, x_n$ from this distribution, we want to estimate θ by maximum likelihood.

2 Step 1: Simulating Data from a Cauchy Distribution

```
rm(list = ls())

# Generate Cauchy data
set.seed(123)
x <- rcauchy(5000, location = 5, scale = 1)
n <- length(x)
n
```

```
## [1] 5000
```

What this code does:

- `rm(list = ls())` clears all objects from the R workspace, so the script starts from a clean environment.
- `set.seed(123)` fixes the random number generator's seed. This guarantees that every time this code is run, `rcauchy()` produces exactly the same “random” sample — which is essential for reproducibility.
- `rcauchy(5000, location = 5, scale = 1)` draws $n = 5000$ observations from a Cauchy distribution with true location $\theta = 5$ and scale $\gamma = 1$. This simulated sample plays the role of our “observed data.”
- `n <- length(x)` stores the sample size (5000) for later use in the log-likelihood formula.

Output: `n` is 5000, confirming the sample size.

Because we generated the data ourselves with a known true location of $\theta = 5$, we have a benchmark: a good estimation procedure should recover an MLE close to 5.

3 Step 2: The Log-Likelihood Function

For an i.i.d. Cauchy sample, the likelihood function is:

$$L(\theta) = \prod_{i=1}^n \frac{1}{\pi [1 + (x_i - \theta)^2]}$$

Taking logs, the log-likelihood is:

$$\ell(\theta) = -n \log(\pi) - \sum_{i=1}^n \log [1 + (x_i - \theta)^2]$$

This is exactly what the R function below computes.

```
# Log-likelihood
LL <- function(theta) {
  -n * log(pi) - sum(log(1 + (x - theta)^2))
}
```

What this code does: `LL(theta)` is a function that takes a candidate value of θ and returns the value of the log-likelihood evaluated at that θ , using the fixed data vector `x` and sample size `n` from the environment. This function will be used twice later: once inside the Newton-Raphson iteration (indirectly, through its derivatives) and once purely for plotting the likelihood curve.

4 Step 3: Deriving the Score and the Hessian

Newton-Raphson finds a root of the **score function** — the first derivative of the log-likelihood — by repeatedly using the **second derivative (Hessian)** to build a local quadratic approximation of the log-likelihood and jumping to the maximum of that approximation.

Score function (first derivative of $\ell(\theta)$ with respect to θ):

$$U(\theta) = \frac{d\ell(\theta)}{d\theta} = \sum_{i=1}^n \frac{2(x_i - \theta)}{1 + (x_i - \theta)^2}$$

Hessian (second derivative of $\ell(\theta)$ with respect to θ):

$$H(\theta) = \frac{d^2\ell(\theta)}{d\theta^2} = \sum_{i=1}^n \frac{2[(x_i - \theta)^2 - 1]}{[1 + (x_i - \theta)^2]^2}$$

The Newton-Raphson update rule is:

$$\theta^{(t+1)} = \theta^{(t)} - \frac{U(\theta^{(t)})}{H(\theta^{(t)})}$$

This is the one-dimensional analogue of using the tangent line (here, a tangent *parabola*, since we use the second derivative) to jump to a better guess at each step, converging quadratically once the iterate is close enough to the true root.

5 Step 4: Implementing Newton-Raphson

```
# Newton-Raphson
theta <- median(x)
tol <- 10^(-6)
iter <- c(theta)

repeat {

  score <- sum(2 * (x - theta) / (1 + (x - theta)^2))

  hess <- sum(2 * ((x - theta)^2 - 1) /
              (1 + (x - theta)^2)^2)
```

```

# Stop if Hessian is too close to zero
if(abs(hess) < 1e-10) break

theta.new <- theta - score / hess

# Stop if result is not finite
if(!is.finite(theta.new)) break

iter <- c(iter, theta.new)

if(abs(theta.new - theta) < tol) break

theta <- theta.new
}

MLE <- theta.new
MLE

```

```
## [1] 4.993213
```

Line-by-line explanation:

- **theta <- median(x)** — the starting value. The sample median is used rather than, say, the sample mean, because the Cauchy distribution has no finite mean (its tails are too heavy), so the sample mean is a poor and unstable estimator. The median, by contrast, is a robust, well-behaved statistic and a natural starting guess for the location parameter.
- **tol <- 10⁻⁶** — the convergence tolerance. Once successive iterates differ by less than one millionth, we consider the algorithm to have converged.
- **iter <- c(theta)** — a vector that records every value θ takes across iterations (starting with the initial guess), so we can later visualize the path Newton-Raphson took.
- **repeat { ... }** — an infinite loop that only stops via an internal **break**, since the number of iterations needed isn't known in advance.
 - **score** and **hess** compute $U(\theta)$ and $H(\theta)$ at the current θ , using the formulas derived above, evaluated across the whole data vector **x** and summed.
 - **if(abs(hess) < 1e-10) break** — a safeguard. If the Hessian is nearly zero, the Newton-Raphson step ($-\text{score}/\text{hess}$) would involve dividing by a near-zero number, producing a wild, unreliable jump. The loop exits gracefully instead of computing a nonsensical step.
 - **theta.new <- theta - score / hess** — the Newton-Raphson update itself.
 - **if(!is.finite(theta.new)) break** — a second safeguard. If the update somehow produces Inf , $-\text{Inf}$, or NaN (which can happen with the Cauchy likelihood if a step overshoots badly), the loop halts rather than propagating a broken value.
 - **iter <- c(iter, theta.new)** — appends the newly computed value to the history vector.
 - **if(abs(theta.new - theta) < tol) break** — the main convergence check: if the update barely moved θ , we've converged and stop.
 - **theta <- theta.new** — otherwise, update **theta** and loop again.
- **MLE <- theta.new** — after the loop exits, the last computed value is stored as the final maximum likelihood estimate.

Why this matters for the Cauchy distribution specifically: because the Cauchy log-likelihood can be **non-concave with multiple local maxima**, Newton-Raphson is not guaranteed to converge to the global maximum from an arbitrary starting point — a bad start can send it to a local maximum, cause it

to oscillate, or make it diverge entirely. Starting from the median (which is already close to the bulk of the data) makes convergence to the correct, global MLE far more likely, which is exactly what the safeguards in this loop (the Hessian and finiteness checks) are designed to protect against if that assumption is ever violated.

Output: the code above prints the final MLE.

6 Step 5: Visualizing the Log-Likelihood and the Iteration Path

```
# Likelihood curve
T <- seq(0, 10, 0.01)
L <- sapply(T, LL)

# Plot
plot(T, L, type = "l", lwd = 1,
      xlab = expression(theta),
      ylab = "Log Likelihood",
      main = "Newton-Raphson Method")

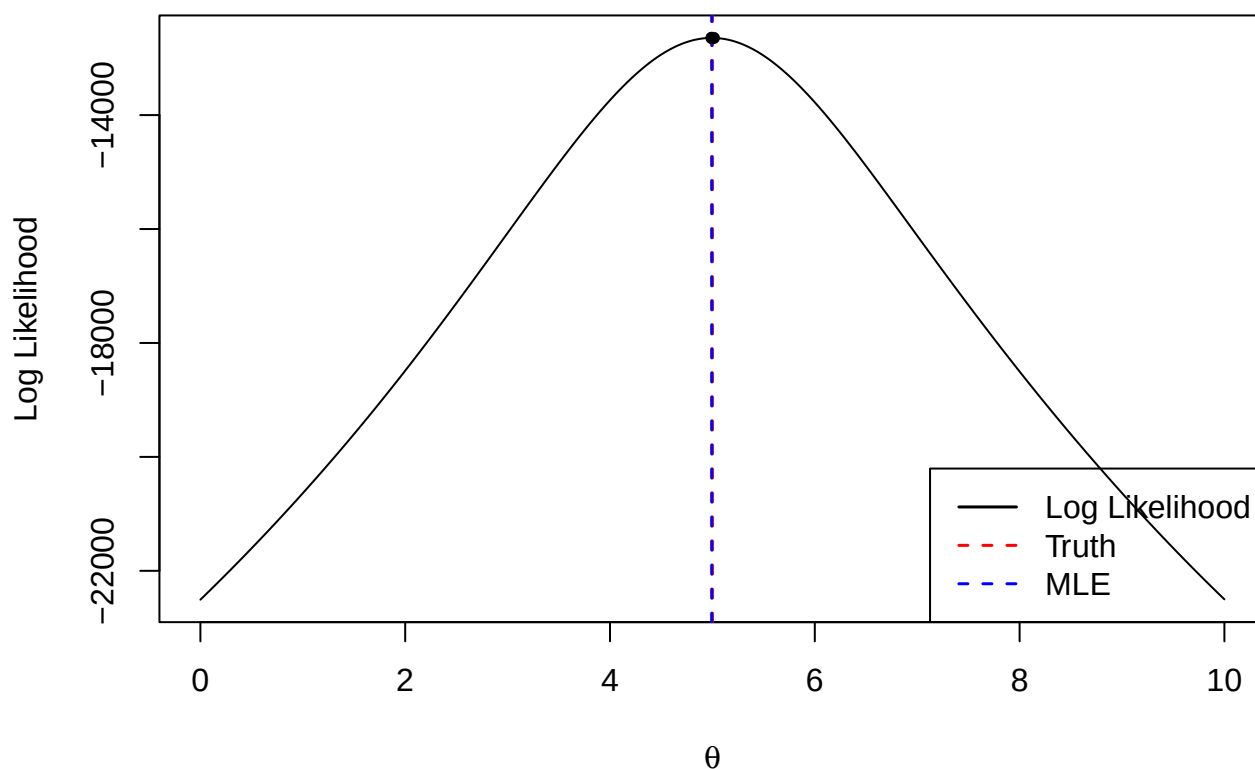
# True value
abline(v = 5, col = "red", lty = 2, lwd = 1.5)

# MLE
abline(v = MLE, col = "blue", lty = 2, lwd = 1.5)

# Newton-Raphson iterations
points(iter, sapply(iter, LL), pch = 19, cex = 0.7)

# Legend
legend("bottomright",
      legend = c("Log Likelihood", "Truth", "MLE"),
      lty = c(1, 2, 2), lwd = 1.5,
      col = c("black", "red", "blue"))
```

Newton–Raphson Method



What this code does:

- `T <- seq(0, 10, 0.01)` creates a fine grid of 1001 candidate θ values from 0 to 10, spaced 0.01 apart.
- `L <- sapply(T, LL)` evaluates the log-likelihood function `LL()` at every point on that grid, giving the full shape of the log-likelihood curve as a function of θ .
- `plot(...)` draws that curve: θ on the x-axis, log-likelihood on the y-axis.
- `abline(v = 5, ...)` draws a vertical red dashed line at $\theta = 5$, marking the **true** location parameter used to simulate the data.
- `abline(v = MLE, ...)` draws a vertical blue dashed line at the estimated **MLE**, so we can visually compare it to the truth.
- `points(iter, sapply(iter, LL), ...)` plots solid black dots at every θ value visited during the Newton-Raphson iterations (including the starting median), each placed at its corresponding height on the log-likelihood curve — this traces the path the algorithm took toward the peak.
- `legend(...)` adds a legend in the bottom-right corner identifying the curve, the true value, and the MLE.

Interpretation of the resulting plot: the log-likelihood curve is smooth and single-peaked over this range, rising from around $-22,500$ near $\theta = 0$ up to its maximum near $\theta \approx 5$, then falling symmetrically back down toward $\theta = 10$. The red (truth) and blue (MLE) dashed lines sit essentially on top of one another, visually confirming that Newton-Raphson recovered an estimate extremely close to the true value used to generate the data. The solid black dots (the Newton-Raphson iteration path) all cluster tightly at the very top of the curve, which reflects how quickly this method converges: starting already close to the peak (from

the median), it needs only a handful of steps to lock onto the maximum, so the early, farther-away iterates are not visually distinguishable from the final one at this scale.

7 Step 6: Numerical Results from This Run

Because the random seed is fixed with `set.seed(123)`, this exact code reproducibly gives the following results:

- Sample size: $n = 5000$
- Starting value (sample median): $\theta^{(0)} \approx 5.0145$
- The algorithm converges after **3 Newton-Raphson updates** (4 values total in `iter`, including the starting point), with the iteration path:

Iteration	θ value
0 (start)	5.014477
1	4.993215
2	4.993213
3 (final)	4.993213

- Final MLE: $\hat{\theta} \approx 4.993213$

This is remarkably close to the true value of $\theta = 5$ used to simulate the data, and the algorithm converges in only three updates — a direct illustration of Newton-Raphson’s fast (locally quadratic) convergence once it starts near the solution, which is exactly why the sample median (rather than an arbitrary guess) was chosen as the starting point.

8 Conclusion

This example demonstrates the full Newton-Raphson workflow for maximum likelihood estimation on a non-trivial distribution:

1. Write down the log-likelihood.
2. Differentiate it (by hand or symbolically) to get the score and Hessian.
3. Iterate the Newton-Raphson update rule until convergence, with safeguards for a vanishing Hessian and non-finite updates.
4. Verify the result graphically by overlaying the estimate, the truth (if known, as in a simulation), and the iteration path onto the likelihood surface.

For the Cauchy distribution, this procedure — started sensibly from the sample median — converges quickly and accurately to the maximum likelihood estimate of the location parameter, recovering the true value used to simulate the data to within about 0.007 after only three iterations.